

Trustworthiness of WCET Analysis Methods in Real-Time and Embedded Systems

Muhammad Umer Hayat
Hochschule Hamm-Lippstadt
muhammad-umer.hayat@stud.hshl.de

Abstract—Worst-Case Execution Time (WCET) analysis is required when software tasks in real-time and safety-critical systems must finish before fixed deadlines. The main challenge is not only to compute a large execution-time value, but to justify that this value can be trusted on the target hardware and for the intended application. This paper discusses the main WCET analysis families: static, measurement-based, hybrid, deterministic, and probabilistic methods. It compares their assumptions, engineering costs, certification relevance, and scalability limits. The central argument is that no WCET method is best in general. Static deterministic timing analysis gives the strongest formal argument when hardware and software are simple and well documented. Measurement-based methods are practical and widely used, but their trustworthiness depends on test coverage and measurement conditions. Hybrid methods improve path coverage but do not remove all hardware-state uncertainty. Probabilistic methods, especially measurement-based probabilistic timing analysis, are promising for complex and multicore platforms, but still face hardware-support and certification barriers. The paper concludes that WCET method selection should be driven by the validity of assumptions on the target system, not by the theoretical strength of the method alone.

Index Terms—Worst-case execution time, WCET, real-time systems, embedded systems, timing analysis, trustworthiness, multicore processors, certification.

I. INTRODUCTION

REAL-TIME systems are not only required to compute correct results. They must also compute those results within bounded time. In a hard real-time system, a late result can be considered a system failure even if the logical value is correct. Examples include flight-control software, railway interlocking, automotive control units, and industrial safety functions. In such systems, engineers need an upper bound on the execution time of each critical task. This upper bound is usually called the Worst-Case Execution Time (WCET).

At a first level, WCET appears to be a numerical problem: determine the largest possible time a program can take on a target platform. In practice, however, WCET analysis is also an evidence problem. The estimate must be high enough to be safe, tight enough to be useful, and supported by assumptions that hold in the real deployed system. A very pessimistic WCET may make the system appear unschedulable. A too optimistic WCET may hide deadline failures.

The paper by Abella et al. is useful because it does not treat WCET methods only as algorithms. It evaluates them by asking whether their assumptions are viable in industrial settings, especially when processor timing behavior is difficult to document or reproduce [1]. This paper builds on that

perspective. It first introduces the WCET problem and method taxonomy, then places WCET analysis in its scientific context, and finally evaluates the strengths and weaknesses of each method family from an embedded-systems engineering point of view.

II. WCET AND ITS ROLE IN REAL-TIME ANALYSIS

A. Basic Timing Problem

For a periodic or sporadic real-time task τ_i , schedulability analysis usually requires an execution-time bound C_i . A simplified timing requirement can be written as

$$C_i \leq D_i, \quad (1)$$

where D_i is the deadline of the task. In full system analysis, C_i is then used by scheduling methods such as Rate Monotonic (RM), Earliest Deadline First (EDF), or response-time analysis. If C_i is wrong, the later scheduling proof is weakened or invalid.

It is useful to distinguish WCET from Worst-Case Response Time (WCRT). WCET concerns the execution time of one task on a target platform. WCRT extends the problem to a full task set and includes scheduling effects such as preemption, blocking, and interference from other tasks. Therefore, WCRT depends on WCET but is not identical to it.

B. Sources of Timing Uncertainty

Execution time depends on more than the source code. Modern processors include caches, pipelines, branch predictors, prefetchers, buses, memory controllers, and sometimes several cores sharing resources. These features improve average performance but reduce timing predictability. A program may execute quickly in one hardware state and slowly in another, even with the same logical input.

For this reason, trustworthy WCET analysis must handle three types of information:

- *Program structure*, including branches, loops, function calls, and infeasible paths.
- *Hardware timing behavior*, including instruction timing, memory access, caches, and pipeline effects.
- *Operational context*, including input data, operating-system effects, other tasks, compiler version, and deployment platform.

III. TAXONOMY OF WCET METHODS

Abella et al. classify WCET methods along two dimensions: deterministic versus probabilistic reasoning, and static versus measurement-based versus hybrid analysis [1]. This gives six method families, summarized in Table I.

Deterministic methods produce a single WCET estimate. Probabilistic methods produce a probabilistic WCET (pWCET), usually interpreted as a bound with an associated probability of exceedance. This distinction matters for certification. A deterministic value is easier to explain in traditional safety cases, while a probabilistic value can express different confidence levels but requires stronger statistical and certification arguments.

IV. SCIENTIFIC CONTEXT AND RELATED APPROACHES

WCET analysis is connected to several neighboring areas in computer science and embedded systems.

A. Scheduling and Response-Time Analysis

Real-time scheduling theory assumes that execution-time bounds are available. In RM, EDF, and fixed-priority response-time analysis, the task execution-time parameter is a basic input. WCET analysis therefore acts as a bridge between low-level processor behavior and high-level timing guarantees.

B. Formal Methods and Abstract Interpretation

Static WCET analysis is closely related to formal methods. Tools for SDTA often use abstract interpretation to over-approximate possible processor and program states [2]. The advantage is soundness: the analysis is designed not to miss possible behaviors. The cost is pessimism and heavy dependence on a correct hardware timing model.

C. Computer Architecture

Processor architecture is central to WCET trustworthiness. A simple in-order microcontroller without caches is much easier to analyze than a high-performance multicore processor with shared cache, hardware prefetching, and out-of-order execution. The gap between performance-oriented hardware design and timing predictability is one of the main reasons WCET analysis remains difficult.

D. Safety Certification

In avionics, automotive, railway, and industrial systems, timing evidence is part of a broader safety argument. Standards and certification practice do not only ask whether an analysis method is theoretically interesting. They ask whether the method, tools, assumptions, and manual steps can be justified for the required integrity level [1].

E. Alternative and Supporting Strategies

Some approaches do not replace WCET analysis but reduce its difficulty. Time-triggered architectures allocate fixed time slots to tasks or buses, reducing timing uncertainty at the cost of flexibility. Resource isolation, such as cache partitioning or memory throttling, limits interference between tasks. Runtime monitoring can detect timing violations during operation, but

it cannot replace offline evidence because it reacts after the risk has already occurred. Machine-learning prediction may help detect trends, but it does not provide the kind of safety guarantee needed in certified hard real-time systems.

V. TRUSTWORTHINESS AND ENGINEERING TRADEOFFS

A. Trustworthiness Is More Than Method Soundness

A method can be mathematically sound and still produce untrustworthy results if its inputs or assumptions are wrong. For SDTA, a sound path analysis cannot compensate for an incorrect timing model. For MBDTA, accurate measurements cannot prove that the worst-case input was tested. For MBPTA, statistical output is only meaningful if the required independence and representativeness assumptions hold.

This leads to a practical principle: the trustworthiness of a WCET estimate is limited by the weakest step in the analysis chain. Manual loop-bound annotations, incomplete hardware documentation, intrusive measurement instrumentation, and platform differences can all reduce confidence.

B. Formal Guarantee Versus Practical Deployability

SDTA gives the strongest formal argument because it attempts to cover all relevant program paths and hardware states through safe abstraction. This is attractive for safety-critical software. However, it becomes expensive or impractical when the processor is complex or poorly documented. MBDTA is easier to deploy because it measures real hardware, but it cannot guarantee that the actual worst case was observed.

C. Precision Versus Analysis Cost

More detailed models can reduce pessimism, but building and validating those models requires time, expertise, and access to hardware details. The same pattern appears in testing: more measurement runs can increase confidence, but the input space of realistic software is often too large for exhaustive testing.

D. Single-Core Versus Multicore

Classical WCET methods work best when the timing behavior of one task can be analyzed mostly in isolation. Multicore processors break this assumption. Shared caches, buses, memory controllers, and interconnects create timing interference between tasks. A task that appears safe during one measurement campaign may experience different delays when co-running tasks change.

VI. CRITICAL EVALUATION OF METHOD FAMILIES

A. Static Deterministic Timing Analysis

SDTA is the strongest method when the hardware is simple and fully documented. It is especially suitable for small safety-critical programs running on in-order microcontrollers or processors designed for predictability. Its main weakness is dependency on the hardware timing model and on program flow facts. If vendor documentation is incomplete, if errata affect timing behavior, or if manual loop annotations are wrong, the formal guarantee becomes fragile. For modern processors, SDTA may also become too pessimistic or too expensive.

TABLE I
MAIN WCET METHOD FAMILIES

Method	Type	Core idea	Main strength	Main limitation
SDTA	Static deterministic	Analyze program paths and hardware timing model without executing the program.	Can provide a mathematically sound deterministic upper bound.	Depends on correct hardware models and correct program flow facts.
MBDTA	Measurement-based deterministic	Execute the program on target hardware and derive a bound from observed times.	Practical and based on real execution data.	The true worst-case path or hardware state may not be tested.
HYDTA	Hybrid deterministic	Combine measurements with static path information.	Better path coverage than pure measurement.	Separate measurements may not preserve real cache or memory state.
SPTA	Static probabilistic	Use probabilistic timing models and static reasoning.	Gives probabilistic arguments without full testing.	Requires simple or analyzable randomized timing models.
MBPTA	Measurement-based probabilistic	Use repeated measurements and statistics to estimate probabilistic WCET.	Promising for complex hardware and mixed-criticality settings.	Requires statistical validity and often hardware time randomization.
HYPTA	Hybrid probabilistic	Combine probabilistic measurements with structural program information.	Reduces measurement effort while keeping probabilistic reasoning.	Less mature and difficult to justify for certification.

TABLE II
ENGINEERING TRADEOFFS OF REPRESENTATIVE WCET METHODS

Method	Trust argument	Cost driver	Best fit	Main risk
SDTA	Formal upper-bound reasoning	Timing model and flow facts	Simple, documented processors	Wrong model or wrong annotations
MBDTA	Real execution observations	Test campaign design	Stable single-core platforms	Worst case remains unobserved
HYDTA	Measurements plus static paths	Combining path and timing evidence	Programs with known structure	Lost cache or memory-state effects
MBPTA	Statistical tail estimation	Valid randomization and many measurements	Research multicore and mixed-criticality systems	Statistical or certification assumptions fail

B. Measurement-Based Deterministic Timing Analysis

MBDTA is attractive because it uses real execution data from the target hardware. It avoids the need to build a full timing model and is therefore often practical in industry. Its weakness is coverage. The highest observed execution time is not necessarily close to the true WCET. Adding a fixed margin over the highest observed time is simple, but the margin has no general scientific guarantee. MBDTA is strongest when the platform is stable, the input space is well understood, and the same kind of measurement process has already been accepted in similar systems.

C. Hybrid Deterministic Timing Analysis

HYDTA attempts to combine the strengths of SDTA and MBDTA. Static path information can improve coverage beyond observed test cases, while measurements keep the method closer to real hardware behavior. This makes HYDTA an important practical compromise. However, it does not fully solve hardware-state uncertainty. Measuring program fragments separately may fail to reproduce cache, branch predictor, or memory-controller states that occur during full execution.

D. Probabilistic Timing Analysis

Probabilistic methods change the form of the result. Instead of one deterministic bound, they estimate a probability that execution time exceeds a certain value. MBPTA is the most promising member of this group because it can use real

measurements and extreme-value reasoning to estimate rare execution-time tails [3]. Its major difficulty is that the statistical model needs suitable conditions. In many proposals, hardware time randomization is required so that measurements become representative and independent enough for the analysis. This hardware support is not yet common in certified production systems.

VII. APPLICATION GUIDANCE

Table III summarizes where the main method families are suitable and where they are weak.

The most important engineering lesson is that method choice should start from the target system. If the processor is simple and documentation is reliable, SDTA may be the best option. If the hardware is stable and well tested but too difficult to model, MBDTA or HYDTA may be more realistic. If high-performance multicore hardware is required, then current deterministic approaches struggle, and probabilistic or resource-isolation approaches become relevant. However, for certified products, these newer approaches still need stronger industrial evidence.

VIII. OPEN RESEARCH QUESTIONS

Several problems remain open.

A. Multicore WCET

The largest gap is multicore timing analysis under realistic workloads. Shared-resource interference is difficult to bound

TABLE III
SUITABLE AND UNSUITABLE APPLICATION SCENARIOS

Method	Suitable scenario	Unsuitable scenario
SDTA	Engine-control or industrial-control software on a simple, documented in-order microcontroller.	Modern application processor with out-of-order execution, shared cache, and limited timing documentation.
MBDTA	Mature single-core railway or industrial platform with stable hardware and strong test history.	Autonomous driving or UAV software with huge input space and changing environments.
HYDTA	Embedded control code where path structure is known but pure testing gives weak coverage.	Software with strong cache and memory-state interactions between many measured segments.
MBPTA	Research or prototype mixed-criticality multicore platform with timing randomization support.	Certified product today without mature hardware support and accepted certification argument.

without being either unsafe or extremely pessimistic. Practical approaches may need to combine timing analysis with architectural mechanisms such as cache partitioning, memory bandwidth control, or time-triggered access to shared resources.

B. Automatic Flow-Fact Extraction

SDTA depends on flow facts such as loop bounds and infeasible paths. Manual annotations are error-prone and expensive. Better automatic extraction would improve both trustworthiness and usability.

C. Certification of Probabilistic Results

Probabilistic WCET can express risk more flexibly than one deterministic value, but certification practice still needs clear rules for interpreting pWCET evidence. The key question is how to argue that a probability of exceedance is low enough for a given safety level.

D. Timing of AI and Accelerator Workloads

Modern embedded systems increasingly include neural-network inference, GPUs, FPGAs, and other accelerators. These workloads create large memory traffic, parallel execution, and data-dependent behavior. Classical CPU-based WCET assumptions do not transfer cleanly to these systems.

E. Online Adaptation

Most WCET methods are offline. Adaptive systems raise the question of whether runtime measurements can update timing evidence safely after deployment. This is difficult because runtime monitoring can observe timing behavior, but it must not hide rare future deadline violations.

IX. CONCLUSION

WCET analysis is essential for real-time and safety-critical embedded systems, but its trustworthiness cannot be judged by method soundness alone. A WCET estimate is only as reliable as the assumptions, models, measurements, annotations, and platform conditions that support it.

SDTA provides the strongest formal argument when hardware and software are simple enough. MBDTA is practical and widely usable, but it depends heavily on test coverage. HYDTA improves measurement-based analysis with static structure, yet still faces hardware-state problems. MBPTA and related probabilistic methods are promising for complex and multicore

systems, but they still require stronger hardware support and accepted certification arguments.

Therefore, there is no universally best WCET method. The correct question is not which method is strongest in theory, but which method's assumptions hold for the target system. As embedded systems move toward high-performance multicore and AI-enabled platforms, closing the gap between hardware complexity and analyzable timing behavior remains the central challenge.

REFERENCES

- [1] J. Abella, C. Hernandez, E. Quinones, F. J. Cazorla, P. R. Conmy, M. Azkarate-Askasua, J. Perez, E. Mezzetti, and T. Vardanega, "WCET Analysis Methods: Pitfalls and Challenges on their Trustworthiness," in *Proc. 10th IEEE International Symposium on Industrial Embedded Systems (SIES)*, Siegen, Germany, 2015, pp. 1–10, doi: 10.1109/SIES.2015.7185039.
- [2] R. Wilhelm et al., "The worst-case execution time problem: Overview of methods and survey of tools," *ACM Transactions on Embedded Computing Systems*, vol. 7, no. 3, pp. 1–53, 2008.
- [3] L. Cucu-Grosjean et al., "Measurement-based probabilistic timing analysis for multi-path programs," in *Proc. Euromicro Conference on Real-Time Systems (ECRTS)*, 2012, pp. 91–101.